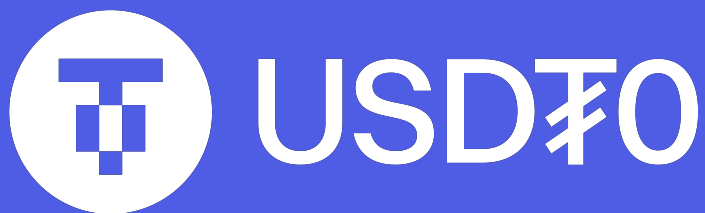


Everdawn USDT0 Hedera Deployment Assessment



February 26, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
Overview	4
Approach	4
USDT0 on Hedera	6
Deployment Overview	6
Conclusion	9
Appendix	10
Issue Classification	10

Summary

Type	Stablecoins	Total Issues	0 (0 resolved)
Timeline	From 2026-02-23 To 2026-02-24	Critical Severity Issues	0 (0 resolved)
Languages	TypeScript	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	0 (0 resolved)

Scope

This report provides a detailed assessment of the deployment scripts and the resulting on-chain deployments of the USDT0 token on the Hedera network.

The deployment was introduced via [pull request #182](#) of the [Everdawn-Labs/usdt0-oft-contracts](#) repository.

Overview

USDT0 is an ERC-20 wrapper for Tether (USDT), designed to integrate with LayerZero's Omnichain Fungible Token (OFT) standard. By leveraging this protocol, the tokens gain native cross-chain functionality, enabling seamless transfers, storage, and usage across multiple blockchain networks without relying on third-party bridges.

This approach ensures consistent liquidity and usability under the Omnichain liquidity standard powered by LayerZero. To realize this vision, Everdawn is deploying USDT0 on the Hedera network, ensuring interoperability and accessibility wherever the tokens are used in existing cross-chain deployments.

Approach

The deployment assessment process consists of a comprehensive series of steps designed to confirm the on-chain correctness of all contracts and their initializations. The following items are verified as part of the assessment:

1. The on-chain bytecode of the proxy, implementation, and admin contracts for both the OFT and token contracts matches the expected bytecode.
2. Proxies point to their intended implementation addresses.
3. ProxyAdmin contracts are properly configured and associated with the respective proxies.

4. The OFT contract is linked to the correct token address and the appropriate LayerZero endpoint configuration for the given chain.
5. Where applicable, scheduled transactions in the Safe related to the setup and configuration of these contracts are reviewed upon request.

Contract	Address
Proxy (<code>HTSConnectorUpgradeable</code>)	<code>0xe3119e23fc2371d1E6b01775ba312035425A53d6</code>
Implementation	<code>0xbD3c51411852A1E3aB545FD0bCaF22a3E32d553e</code>
DefaultProxyAdmin	<code>0xDCee5eF6779567C27e69D57f6A143449372D1784</code>
Safe Multisig	<code>0x2BA553f06b26EfE2A3EdDC4f3E20C7Db9B09797f</code>
LZ Endpoint V2 (Hedera)	<code>0x3A73033C0b1407574C76BdBAC67f126f6b4a9AA9</code>
Deployer (bootstrapper EOA)	<code>0x1a6362AD64ccFF5902D46D875B36e8798267d154</code>

More information can be found in the sections below.

USDT0 on Hedera

Deployment scripts for the Hedera network were reviewed at [pull request #182](#) of the [Everdawn-Labs/usdt0-oft-contracts](#) repository.

Deployment Overview

Scripts

- The Hedera chain configuration as a LayerZero endpoint has been added properly ([hardhat.config.ts](#)):
 - The RPC URL is hardcoded in the configuration ([https://tame-ultra-wind.hedera-mainnet.quiknode.pro/...](#)) instead of being sourced from an environment variable. This is a minor credential hygiene concern and the URL should be moved to an environment variable in a future update.
 - The `safeUrl` and `safeAddress` fields both contain `// TBD` comments as of the merge commit, indicating that the Safe integration configuration was not finalized at the time of merge.
- 20 peer connections have been added, pairing Hedera with all existing supported chains ([layerzero-prod.config.ts](#)):
 - All connections correctly specify `['LayerZero Labs', 'USDT0']` as DVNs on the source chain side. The Hedera-side DVN array is empty across all peer entries, consistent with Hedera's HTS connector architecture.
 - The `hederaContract` endpoint is correctly defined using `EndpointId.HEDERA_V2_MAINNET` with the contract name `HTSConnectorUpgradeable`.
 - The `HEDERA_BLOCKS = 60` constant is consistent with Hedera's ~3-second block time.
 - The owner and delegate of the `hederaContract` are both set to `0x2BA553f06b26EfE2A3EdDC4f3E20C7Db9B09797f`, which is consistent with the Safe address in [hardhat.config.ts](#).
- The post-deploy token association and minting has been commented out, which should be confirmed as intentional. ([deploy/HTSConnectorUpgradeableDeploy.ts](#))

- The `updateAdminKey.ts` script has been updated to target Hedera mainnet with the following hardcoded values: account ID `0.0.9470763`, Safe wallet `0.0.9938995`, and token ID `0.0.10282787`.
 - The `myAccountId` variable is hardcoded instead of being sourced from an environment variable. This is a minor credential hygiene concern.
 - There is no check preventing accidental execution against hedera-mainnet. Deployment scripts operating on the mainnet should require explicit confirmation or an environment flag, consistent with the pattern used in other deployment scripts in the repository.

On-Chain Deployments

The proxy follows the standard OpenZeppelin TransparentUpgradeableProxy pattern:

```
Proxy (0xe311) → Implementation (0xbD3c) via EIP-1967 slot
ProxyAdmin (0xDCee) → owner: Safe (0x2BA5)
Proxy EIP-1967 admin slot → ProxyAdmin (0xDCee)
```

The initialization parameters:

```
name: USDT0
symbol: USDT0
adminKey: 0x032a...e6b2b (33-byte SECP256K1)
_delegate: 0x1a63...7d154 (bootstrapper EOA at deploy time)
_safeWalletAddr: 0x2BA553f06b26EfE2A3EdDC4f3E20C7Db9B09797f
```

Note that the `_delegate` parameter was set to the bootstrapper EOA at initialization. The wiring batch resolves this by calling `setDelegate(Safe)` as the final transaction.

The on-chain state summary is also provided below:

Check	Expected	Result
ProxyAdmin.owner()	Safe (0x2BA5)	✓ Confirmed
proxy.owner()	Safe (0x2BA5)	✓ Confirmed
proxy EIP-1967 admin slot	ProxyAdmin (0xDCee)	✓ Confirmed (AdminChanged log)
implementation._disableInitializers	Initialized(uint64.max)	✓ Confirmed
LZ endpoint address (immutable)	0x3A73...9AA9	✓ Matches LZ Hedera docs

Check	Expected	Result
endpoint.delegates(proxy)	Safe (0x2BA5)	⚠ Unset — pending wiring batch execution
proxy.htsTokenAddress()	Non-zero (post-createHTSToken)	✗ 0x0 — token not created
Safe threshold	3-of-5	✗ 1-of-5 — nonce 8 pending
Bytecode verification	Matches artifact deployedBytecode	✓ Confirmed

Deployment parameters were set correctly, with pending token creation and Safe threshold update to 3-of-5.

The wiring batch ([wiring_batches/hedera-1.json](#)) was reviewed. The batch configures 20 destination pathways from the Hedera OApp ([0xe311](#)) via the LZ endpoint ([0x3A73](#)), covering peer addresses, send/receive libraries, DVN configuration, execution settings, enforced options, and delegate assignment. All transactions target the Hedera source chain (EID 30316). All peer addresses were verified against canonical USDT0 deployment addresses, and infrastructure addresses are consistent across all 20 pathways.

A bytecode-level comparison was performed for the deployment included in [pull request #144](#) at [commit 3e7a0bf](#). The verifications found no discrepancies.

Conclusion

OpenZeppelin reviewed the scripts that deploy the USDT0 token on the Hedera network. Overall, everything appears to be in order. Subject to the execution of nonce 8 (3-of-5 threshold update), the wiring batch, and HTS token creation call, the deployment is ready for production activation. The audit team is grateful to Everdawn as a partner and looks forward to their project's success.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.